Microservices Topics to Learn for Interview

A Complete Guide for Java Developers

Page One

1 Microservices Fundamentals



- What are Microservices?
- Monolith vs Microservices
- Advantages and Disadvantages
- When to use Microservices?
- Key Characteristics
- Common Use Cases
- Challenges in Microservices
- Best Practices

2 Design & Architecture



- Microservices Architecture Principles
- Domain Driven Design (DDD)
- Bounded Context
- Service Decomposition Strategies
- Database per Service
- Synchronous vs Asynchronous Communication
- Event Driven Architecture
- 12-Factor App Principles

3 Spring Boot for Microservices



- Spring Boot Fundamentals
- Creating Microservices with Spring Boot
- Spring Boot Starters
- Externalized Configuration
- Profiles and Environment Management
- Spring Boot Actuator
- Spring Boot DevTools
- Packaging and Deployment
- Best Practices

4 Service Communication



- REST APIs
- gRPC (Basics)
- Synchronous vs Asynchronous Calls
- Inter-service Communication Patterns
- Request/Response
- Event-driven Communication
- API Gateway Pattern (Intro)
- Handling Timeouts and Retries
- Best Practices

5 Service Discovery



- What is Service Discovery?
- Eureka (Netflix)
- Consul
- Zookeeper (Basics)
- Client-side vs Server-side Discovery
- Service Registration and Heartbeat
- Self-healing and Fault Tolerance
- Integration with Spring Cloud
- Best Practices

6 API Gateway



- What is an API Gateway?
- Role and Benefits
- Spring Cloud Gateway
- Routing and Filtering
- Request/Response Transformation
- Authentication and Authorization
- Rate Limiting
- Logging and Monitoring
- API Gateway vs Load Balancer
- Best Practices

Microservices Topics to Learn for Interview

A Complete Guide for Java Developers

Page Two

7 Resilience & Fault Tolerance

- Why Resilience is important?
- Circuit Breaker (Resilience4j)
- Retry Pattern
- Bulkhead Pattern
- Timeout Handling
- Rate Limiting
- Fallback Mechanism
- Graceful Degradation
- Self-healing Services
- Best Practices

8 Configuration Management

- Externalized Configuration
- Spring Cloud Config Server
- Config Client (Git based configuration)
- Environment-specific Configurations (dev, test, prod)
- Dynamic Configuration Refresh
- Encrypting Sensitive Properties
- Using Vault (HashiCorp)
- Property Source Precedence
- Best Practices

9 Security

- Authentication vs Authorization
- OAuth2 and OpenID Connect
- JWT (JSON Web Token)
- Keycloak Integration
- Spring Security for Microservices
- Role-based Access Control (RBAC)
- Method Level Security
- Securing REST APIs
- Inter-service Security (mTLS, Tokens)
- Best Practices

10 Data Management in Microservices

- Database per Service Pattern
- Relational vs NoSQL Databases
- Schema Design and Evolution
- Data Consistency Challenges
- Saga Pattern (Distributed Transactions)
- Eventual Consistency
- CQRS (Command Query Responsibility Segregation)
- Handling Transactions Across Services
- Data Migration Strategies
- Best Practices

11 Event-Driven Architecture

- Why Event-Driven Architecture?
- Apache Kafka Fundamentals
- Kafka Topics, Partitions and Offsets
- Producers and Consumers
- Consumer Groups
- Exactly Once vs At-Least-Once
- Event Schema and Serialization (Avro/JSON)
- Event Sourcing Basics
- Handling Failures and Retries
- Best Practices

12 Testing in Microservices

- Unit Testing with JUnit and Mockito
- Integration Testing with TestContainers
- Contract Testing (Spring Cloud Contract)
- End-to-End Testing
- Testing REST APIs (WebTestClient)
- Testing with Embedded Databases
- Mocking External Services (WireMock)
- Test Data Management
- Code Coverage and Quality Checks
- Best Practices

Microservices Topics to Learn for Interview

A Complete Guide for Java Developers

Page Three

13 Containerization & Orchestration



- Why Containerization is important?
- Docker Basics (Images, Containers, Dockerfile)
- Building and Running Spring Boot Application with Docker
- Multi-stage Dockerfile
- Docker Compose for Multi-container Setup (e.g., App + DB)
- Kubernetes Basics (Pods, Deployments, Services, ConfigMaps, Secrets)
- Helm Charts (Basics)
- Scaling and Self-healing in Kubernetes
- Best Practices

14 Deployment & CI/CD



- Deployment Strategies (Blue/Green, Rolling, Canary)
- CI/CD Pipeline (Jenkins/GitHub Actions/ GitLab CI)
- Build, Test and Deploy Pipeline
- Environment Management (Dev, Test, Staging, Prod)
- Infrastructure as Code (Terraform Basics)
- Automated Rollbacks
- Container Registry (Docker Hub, ECR)
- Monitoring Deployment (Health Checks)
- Best Practices

15 Monitoring & Observability



- Why Monitoring is important?
- Application Monitoring (Spring Boot Actuator)
- Metrics (Prometheus)
- Visualization (Grafana)
- Logging (ELK Stack – Elasticsearch, Logstash, Kibana)
- Distributed Tracing (Sleuth + Zipkin)
- Alerting (AlertManager)
- Health Checks and Custom Metrics
- Centralized Logging for Microservices
- Best Practices

16 Messaging & Asynchronous Communication



- Why Asynchronous Communication?
- Message Brokers (Kafka, RabbitMQ)
- Kafka Fundamentals (Topics, Partitions, Producers, Consumers)
- RabbitMQ Basics (Exchanges, Queues, Bindings)
- Event-Driven Architecture
- Exactly Once vs At-Least-Once Delivery
- Dead Letter Queue (DLQ)
- Event Sourcing and CQRS (Intro)
- Handling Failures and Retries
- Best Practices

17 Caching in Microservices



- Why Caching is important?
- Caching Strategies (Cache-Aside, Write-Through, Write-Behind)
- Distributed Caching (Redis)
- Cache Invalidation Techniques
- Caching at Different Layers (Application, API Gateway, Database)
- Handling Cache Consistency
- TTL and Expiration Policies
- Use Cases (Read-heavy Services)
- Best Practices

18 Real-World Scenarios & System Design



- Design a Microservices-based E-commerce Application
- Design URL Shortening Service
- Design a Real-time Notification System
- Design a Payment Service
- Design a Ride Booking System (High Level)
- Handling High Traffic and Scalability
- Data Partitioning and Sharding (Basics)
- Trade-offs and Decision Making
- Common Interview Questions and Answers
- Best Practices